

**ACROPOLIS INSTITUTE OF TECHNOLOGY & RESEARCH,
INDORE**

Department of Information Technology

Academic Year 2025 – 2026

PROJECT REPORT
ON
DYNAMIC TEXT-TO-SQL GENERATOR USING LARGE
LANGUAGE MODELS AND RUNTIME SCHEMA AWARENESS

Submitted in partial fulfilment of the requirements for the Degree of
Bachelor of Engineering in Information Technology

Submitted By

Krish Chourasia (0827IT231069)

Lakshya Bhagwat (0827IT231079)

Mansij Namdev (0827IT231082)

Manthan Jadhav (0827IT231083)

Under the Guidance of

Prof. Kapil Sahu

Department of Information Technology

Acropolis Institute of Technology & Research, Indore

Rajiv Gandhi Proudhyogiki Vishwavidyalaya, Bhopal

CERTIFICATE

This is to certify that the project report entitled

"DYNAMIC TEXT-TO-SQL GENERATOR USING LARGE LANGUAGE MODELS AND RUNTIME SCHEMA AWARENESS"

has been carried out by **Krish Chourasia (0827IT231069), Lakshya Bhagwat (0827IT231079), Mansij Namdev(0827IT231082), Manthan Jadhav (0827IT231083)** under the supervision of **Prof. Kapil Sahu**, Department of Information Technology, Acropolis Institute of Technology & Research, Indore, in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Information Technology, affiliated to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal.

This work is the result of the students' own investigation and, to the best of my knowledge, it contains no material previously published or written by another person, nor any material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement is made in the text.

Project Guide: Prof. Kapil Sahu

Signature:

Date:

Head of Department

Department of Information Technology

Acropolis Institute of Technology & Research, Indore

DECLARATION

We, the undersigned, students of the Department of Information Technology, Acropolis Institute of Technology & Research, Indore, hereby declare that the project entitled:

"DYNAMIC TEXT-TO-SQL GENERATOR USING LARGE LANGUAGE MODELS AND RUNTIME SCHEMA AWARENESS"

is our original work, carried out under the guidance of Prof. Kapil Sahu during the academic year 2025–2026. This project has not been submitted elsewhere for the award of any degree or diploma. All sources of information have been duly cited and acknowledged.

Krish Chourasia [0827IT231069]

Lakshya Bhagwat [0827IT231079]

Mansij Namdev [0827IT231082]

Manthan Jadhav [0827IT231083]

Date:

Place: Indore

ACKNOWLEDGEMENT

We take immense pleasure in expressing our sincere gratitude to all those who extended their support, guidance, and encouragement throughout the development of this project.

We are deeply grateful to Prof. Kapil Sahu, our project guide, for his invaluable mentorship, technical insights, and continuous motivation throughout every phase of this work. His experience and academic rigor have significantly shaped the quality and direction of this project.

We would like to extend our heartfelt thanks to the Head of the Department of Information Technology, Acropolis Institute of Technology & Research, Indore, for providing the necessary academic infrastructure and a conducive environment for research and development.

We are also thankful to all faculty members of the IT Department whose lectures, discussions, and feedback have enriched our understanding of software engineering, database systems, and artificial intelligence.

Our sincere appreciation goes to the open-source community and research scholars whose publications and tools — including Spring Boot, React, SQLite, the Hugging Face Inference API, and the DeepSeek language model — have been instrumental in making this system a reality.

Finally, we owe our deepest gratitude to our families and friends for their unconditional support, patience, and encouragement throughout this academic endeavour.

Krish Chourasia (0827IT231069), Lakshya Bhagwat (0827IT231079), Mansij
Namdev (0827IT231082), Manthan Jadhav (0827IT231083)

Department of Information Technology

ABSTRACT

Relational databases form the foundation of modern data-driven systems; however, querying them requires proficiency in Structured Query Language (SQL), a skill that remains out of reach for most non-technical professionals. This creates a persistent data-access bottleneck in organizations, where business analysts, operations personnel, and researchers are perpetually dependent on developers to retrieve information. Natural Language Processing (NLP) and Large Language Models (LLMs) have opened a pathway to bridge this gap through Text-to-SQL systems — applications capable of translating plain English questions into executable SQL queries.

This project presents the design, development, and evaluation of a Dynamic Text-to-SQL Generator that enables users to query a relational database using natural language without any knowledge of SQL. The system is built upon a three-stage pipeline: runtime schema extraction from a live SQLite database, LLM-driven SQL generation using DeepSeek-V3 via the Hugging Face Inference API, and direct query execution followed by structured result delivery. The defining architectural innovation is runtime schema awareness — the database structure is read dynamically from the system metadata on each request, ensuring that the LLM always receives a current, accurate representation of the schema, eliminating brittleness caused by hardcoded schema definitions.

The system is implemented as a full-stack web application comprising a React 18 frontend and a Spring Boot 3 backend microservice. A pre-loaded SQLite database modelling an e-commerce domain with five interrelated tables serves as the query target. Experimental results demonstrate that the system accurately handles a broad spectrum of query types — including simple lookups, multi-condition filters, multi-table joins, aggregations, and date-range queries — with end-to-end latency between two and six seconds. The generated SQL is surfaced transparently to the user alongside query results, enabling verification and trust.

This work contributes to the growing field of human-database interaction by demonstrating a production-viable architecture that combines modern LLM capabilities

with runtime database introspection, achieving adaptability, transparency, and correctness without reliance on fixed schema embeddings or query templates.

Keywords: Text-to-SQL, Natural Language Processing, Large Language Models, DeepSeek-V3, Runtime Schema Awareness, Spring Boot, React, SQLite, Hugging Face Inference API, Database Query Generation.

TABLE OF CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Tables	vi
Chapter 1. Introduction	1
1.1. Background	1
1.2. Emergence of Large Language Models	1
1.3. Problem of Static Schema Representation	2
1.4. Project Objective	2
1.5. Scope of the Report	2
Chapter 2. Problem Statement and Objectives	3
Chapter 3. Literature Review	5
Chapter 4. Existing System	7
Chapter 5. Proposed System	9
Chapter 6. Proposed Methodology	10
Chapter 7. System Architecture	12
Chapter 8. Technology Stack	13
Chapter 9. Database Design	16
Chapter 10. Module Description	18
Chapter 11. Frontend, Backend, and API Design	20
Chapter 12. Runtime Schema Extraction Process	22
Chapter 13. Natural Language to SQL Conversion Process	23
Chapter 14. Query Execution Pipeline	25
Chapter 15. Implementation Details	26
Chapter 16. Results and Discussion	28
Chapter 17. Advantages, Limitations, and Future Scope	30
Chapter 18. Conclusion	33
References	34

LIST OF TABLES

TABLE I	Literature summary and identified gaps	Pg no.6
TABLE II	Comparison of existing system approaches	Pg no.8
TABLE III	System component and Responsibilities	Pg no.12
TABLE IV	Technology stack summary	Pg no.15
TABLE V	Data base Tables and attributes	Pg no.17
TABLE VI	API endpoint specification	Pg no.21
TABLE VII	Natural language query types and SQL examples	Pg no.24
TABLE VIII	Key maven dependencies	Pg no.27
TABLE IX	Query accuracy dependencies	Pg no.29
TABLE X	System latency measurements	Pg no.29

CHAPTER 1: INTRODUCTION

1.1 Background

The exponential growth of digital data has made relational database management systems (RDBMS) indispensable across industries. Virtually every enterprise, institution, and digital platform relies on structured databases to store, manage, and retrieve critical information. Despite the ubiquity of these systems, access to data remains constrained by a significant technical barrier: the requirement of Structured Query Language (SQL) proficiency.

SQL is a declarative, domain-specific language designed for querying and manipulating relational data. While powerful and expressive, it demands precision in syntax, a thorough understanding of database structure (schema), and an appreciation for relational algebra concepts such as joins, aggregations, and subqueries. These requirements exclude the vast majority of potential data consumers — business analysts, operations managers, healthcare practitioners, educators, and researchers — from directly accessing the data they need.

The consequences of this exclusion are significant. Data bottlenecks form when non-technical users must route all information requests through developers or data engineers. Decision-making slows. The value embedded in organizational data goes partially unrealized. This tension between data abundance and data accessibility has motivated decades of research into natural language interfaces for databases (NLIDB).

1.2 Emergence of Large Language Models

The rapid advancement of Large Language Models (LLMs) — deep neural networks trained on massive corpora of text and code — has fundamentally altered the landscape of Text-to-SQL research. Models such as GPT-4, Llama 3, and DeepSeek-V3 demonstrate an unprecedented capacity to understand natural language intent and generate syntactically correct, semantically meaningful SQL when provided with appropriate context. This capability represents a qualitative leap beyond earlier rule-based and statistical NLP approaches.

However, leveraging LLMs effectively for SQL generation introduces a new challenge: schema awareness. An LLM generates SQL based on patterns learned during training

and on context provided in the input prompt. Without an accurate, up-to-date description of the target database's structure, the model may hallucinate table names, invent column names that do not exist, or produce queries that are syntactically correct but semantically wrong. The quality of LLM-generated SQL is, therefore, tightly coupled to the quality of the schema information supplied to it.

1.3 Problem of Static Schema Representation

Most existing Text-to-SQL implementations resolve the schema-context problem by hardcoding the database schema into the application — embedding table definitions as static strings, configuration files, or prompt templates. This approach is simple but brittle. When the database schema evolves — as schemas routinely do during application development and maintenance — the hardcoded representation becomes stale. The LLM then generates SQL against an outdated schema, producing errors or incorrect results. Every schema change requires a corresponding update to the application code, creating a tight coupling between the data layer and the AI layer that undermines maintainability.

1.4 Project Objective

This project addresses this limitation through a fundamentally different approach: runtime schema extraction. Rather than embedding a static schema representation, the system reads the database's own internal metadata — its live schema — at the time of each user request. The LLM is thus always provided with an accurate, current description of the database, regardless of any structural changes made since the application was deployed.

The result is a Dynamic Text-to-SQL Generator: a full-stack web application that allows users to pose natural language questions about a relational database and receive structured query results, with the SQL generation process driven by DeepSeek-V3 (accessed via the Hugging Face Inference API) and grounded in a freshly extracted schema on every request.

1.5 Scope of the Report

This report provides a comprehensive account of the project, covering the problem domain, a review of relevant prior work, the proposed system design, the architectural and technology decisions made, the implementation details, and an evaluation of system performance. It is organized to provide both breadth (for readers seeking a high-level overview) and depth (for readers interested in technical specifics of the NLP pipeline, schema extraction process, or technology stack).

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

The proliferation of data in modern organizations has not been accompanied by a proportional democratization of data access. Relational databases — the dominant storage and retrieval paradigm — require SQL proficiency that is confined to a small subset of technical staff. This disparity creates a structural impediment to data-driven decision-making at all organizational levels.

Existing Text-to-SQL solutions suffer from one or more of the following limitations:

- **Static schema dependency:** Schema information is hardcoded into the application, requiring manual updates with every schema change and leading to mismatches between the LLM's schema context and the actual database structure.
- **Low adaptability:** Systems trained or configured for a specific schema cannot be repurposed without significant re-engineering.
- **Opacity of operation:** Many systems conceal the generated SQL from users, making it impossible to verify whether the query accurately represents the user's intent.
- **Limited query scope:** Template-based systems match user questions against a fixed library of patterns, failing on queries that fall outside the template set.

This project addresses these limitations by designing a system that (i) extracts schema information at runtime rather than relying on static embeddings, (ii) employs a capable LLM to handle arbitrary natural language questions, (iii) surfaces the generated SQL transparently to the user, and (iv) is architecturally decoupled so that the AI layer and the data layer can evolve independently.

2.2 Objectives

The primary objectives of this project are as follows:

1. To design and implement a three-stage Text-to-SQL pipeline consisting of runtime schema extraction, LLM-based SQL generation, and direct query execution.
2. To achieve runtime schema awareness by reading the live database metadata on every user request, ensuring the LLM always has an accurate and current representation of the schema.
3. To integrate DeepSeek-V3 via the Hugging Face Inference API as the SQL generation engine, leveraging its advanced code generation and reasoning capabilities.
4. To build a full-stack web application with a React frontend and a Spring Boot backend that is responsive, intuitive, and accessible to non-technical users.
5. To support a wide spectrum of query types, including simple projections, filtered lookups, multi-table joins, aggregation functions, and date-range queries.
6. To surface the generated SQL to the user alongside query results, enabling transparency and verification.
7. To evaluate the system's correctness, robustness, and latency across a representative set of natural language queries.
8. To identify limitations and propose directions for future enhancement toward production deployment.

CHAPTER 3: LITERATURE REVIEW

3.1 Natural Language Interfaces for Databases

The concept of enabling non-technical users to query databases using natural language has been studied since the 1970s. Early systems such as LUNAR [1] and BASEBALL [2] used hand-crafted grammars and domain-specific lexicons to translate English questions into database queries. These systems were brittle, domain-locked, and required extensive manual engineering for each new application domain. Subsequent decades saw improvements through semantic parsing approaches, which attempted to build formal logical representations of natural language questions that could be compiled into SQL.

The introduction of statistical and neural approaches to NLP in the 2010s opened new possibilities. Seq2Seq neural architectures — initially developed for machine translation — were adapted for the Text-to-SQL task, training models to directly map input utterances to SQL strings. Benchmark datasets such as WikiSQL [3] and Spider [4] catalysed substantial research progress. The Spider benchmark, in particular, introduced cross-domain generalization as a key evaluation criterion, requiring systems to generate SQL for unseen database schemas, which forced research attention onto the schema-linking problem.

3.2 Schema Linking and Representation

Schema linking — the process of mapping tokens in a natural language question to the relevant tables, columns, and values in the target schema — has emerged as one of the central challenges in Text-to-SQL research. IRNet [5] introduced intermediate representation to bridge the gap between natural language and SQL syntax. RAT-SQL [6] proposed relation-aware self-attention to capture the structural relationships between question tokens and schema elements. BRIDGE [7] explored schema serialization strategies for feeding schema information to pre-trained language models. These works collectively establish that the accuracy of SQL generation is

fundamentally dependent on the quality and freshness of the schema information presented to the model.

3.3 Large Language Models for Code and SQL Generation

The emergence of GPT-3 [8] and subsequent LLMs fundamentally changed the Text-to-SQL paradigm. These models, trained on vast corpora that include SQL code, programming documentation, and structured data, acquire implicit knowledge of SQL syntax, relational patterns, and common query structures without task-specific supervision. DIN-SQL [9] demonstrated that chain-of-thought prompting combined with schema decomposition could achieve state-of-the-art performance on Spider using GPT-4. CodeX [10] and similar code-specialist models showed that LLMs with strong code generation capability significantly outperform general-purpose language models on SQL generation tasks.

DeepSeek-V3 [11], the model used in this project, represents a contemporary high-performance LLM with demonstrated strength in code and SQL generation. Its mixture-of-experts architecture and extensive training on programming corpora make it particularly well-suited for the SQL generation task.

3.4 Prompt Engineering for SQL Generation

Prompt engineering — the design of input context provided to an LLM to elicit desired outputs — plays a crucial role in LLM-based Text-to-SQL systems. Research has established that including the full DDL (Data Definition Language) statements of relevant tables, example rows, and explicit instructions about output format substantially improves SQL accuracy [9]. This finding directly motivates the design decision in this project to include the complete, dynamically extracted schema in every LLM prompt. The system constructs a structured prompt containing the schema, the user's question, and explicit directives for output format, enabling the model to generate well-formed, schema-consistent SQL.

3.5 Runtime Schema Awareness and Dynamic Systems

To the authors' knowledge, existing public research and deployed systems predominantly rely on static schema representations. Systems such as Vanna.AI and similar commercial tools allow users to configure a schema representation that is then stored and reused. While more maintainable than fully hardcoded approaches, these systems still require an explicit synchronization step when the schema changes. The present system's approach of reading the schema from the SQLite system table (sqlite_master) on every request — requiring no synchronization and guaranteeing currency — represents a distinct architectural contribution.

3.6 Literature Gaps

TABLE I. Literature Summary and Identified Gaps

Study / System	Approach	Identified Gap
IRNet (2019)	Intermediate representation for SQL generation	Requires static schema annotation
RAT-SQL (2020)	Relation-aware attention for schema linking	Schema must be pre-indexed; no runtime extraction
DIN-SQL (2023)	Chain-of-thought prompting with GPT-4	Schema supplied statically in prompt templates
BRIDGE (2020)	Schema serialization for pre-trained LMs	Evaluated on fixed benchmark schemas only
Vanna.AI (Commercial)	LLM SQL generation with schema configuration	Schema synchronization required on schema changes
This Work	Runtime schema extraction from live DB metadata	Addresses dynamic schema evolution without code changes

CHAPTER 4: EXISTING SYSTEM

4.1 Overview of Existing Text-to-SQL Approaches

Text-to-SQL systems have evolved through several distinct generations. Understanding these generations and their respective limitations provides the context within which the proposed system's design decisions can be appreciated.

4.2 Rule-Based and Template-Based Systems

Early NLIDB systems operated through handcrafted rules and template matching. A system developer would enumerate a finite set of question patterns and manually author corresponding SQL templates. When a user submitted a query, the system would match the input against the template library, extract slot values, and fill the SQL template. These systems were deterministic and predictable but were entirely incapable of handling queries outside the predefined template set. Any evolution in the database schema or user query vocabulary required manual updates to the rule set.

4.3 Statistical and Early Neural Approaches

Statistical NLP approaches applied probabilistic models — conditional random fields, n-gram language models, and later Seq2Seq networks — to learn SQL generation from question-SQL parallel corpora. These approaches offered broader coverage than rule-based systems but required large labelled datasets for training and struggled with schema generalization. A model trained on one database schema performed poorly on an unseen schema without retraining or fine-tuning, limiting their practical applicability.

4.4 Pre-trained Language Model Approaches

The introduction of BERT, RoBERTa, and T5 as backbone encoders for Text-to-SQL systems dramatically improved performance on established benchmarks. Systems such as GRAPPA and GAP pre-trained transformers specifically on SQL-augmented

corpora to develop schema-awareness. However, these systems still required the schema to be supplied in a preprocessed, static format. The schema was embedded into the model's input as a fixed context, with no mechanism to refresh it at inference time.

4.5 Drawbacks of Existing Systems

TABLE II. Comparison of Existing System Approaches

Approach	Schema Handling	Query Flexibility	Schema Evolution Support
Rule/Template-Based	Hardcoded	Low (fixed patterns)	Manual re-coding required
Statistical NLP	Static training data	Medium	Retraining required
Pre-trained LM (BERT/T5)	Static prompt context	High on benchmark schemas	Manual prompt update required
LLM with static schema	Static string in prompt	Very High	Manual update required
This System (Dynamic)	Runtime extraction from DB	Very High	Automatic – zero code changes

The fundamental limitation shared across all existing categories is the dependence on a schema representation that is decoupled from the live database and therefore susceptible to becoming stale. This limitation becomes increasingly problematic in agile development environments where database schemas change frequently.

CHAPTER 5: PROPOSED SYSTEM

5.1 System Overview

The proposed system is a Dynamic Text-to-SQL Generator implemented as a full-stack web application. It enables users to submit natural language questions via a React-based web interface and receive structured query results derived from a live SQLite database. The distinctive architectural feature of the system is its runtime schema extraction mechanism: rather than relying on any static representation of the database schema, the backend microservice reads the schema directly from the database's internal metadata on every request.

The system eliminates the schema staleness problem entirely. A developer may add a table, rename a column, or restructure the database at any time; the next request to the system will automatically use the updated schema without any application code changes.

5.2 Key Innovations

The proposed system introduces the following key innovations over existing approaches:

- **Runtime Schema Awareness:** Schema is always extracted fresh from `sqlite_master`, eliminating any possibility of schema-context mismatch between the LLM and the actual database.
- **Zero Schema Maintenance:** No schema synchronization, configuration update, or code change is required when the database evolves.
- **State-of-the-Art LLM Integration:** DeepSeek-V3, a frontier model with exceptional SQL and code generation capability, is integrated via the Hugging Face Inference API using the Chat Completions protocol, making the AI layer easily substitutable.
- **Transparent Query Surfacing:** The generated SQL is displayed to the user alongside the results, enabling verification and building user trust.

- **Clean Microservice Architecture:** The system decomposes naturally into a stateless REST API backend and a React SPA frontend, connected through a single well-defined API endpoint.

5.3 Advantages Over Existing Systems

Compared to existing Text-to-SQL systems, the proposed system offers the following advantages:

9. **Schema currency is guaranteed:** The LLM always reasons over the current database structure.
10. **Maintenance cost is reduced:** Schema evolution requires no application-level intervention.
11. **Query coverage is broad:** Arbitrary natural language questions are handled, not just a predefined template set.
12. **The AI and data layers are decoupled:** The LLM provider and the database can each be replaced independently.
13. **Transparency is prioritised:** Users can inspect the generated SQL to verify correctness.

CHAPTER 6: PROPOSED METHODOLOGY

6.1 Development Approach

The system was developed following an iterative and incremental methodology, allowing each functional component to be designed, implemented, and tested independently before integration. This approach aligned naturally with the system's modular architecture, in which the frontend, backend, database layer, and AI integration function as loosely coupled components with well-defined interfaces.

Development proceeded through the following phases:

14. Requirements analysis and schema design — defining the query domain, the database structure, and the user experience requirements.
15. Backend microservice development — implementing the Spring Boot REST API, the schema extraction module, the LLM integration client, and the query execution engine.
16. AI integration — designing and testing the prompt engineering strategy and integrating the Hugging Face Inference API client.
17. Frontend development — building the React SPA with the landing page, the query interface, and the results table component.
18. Integration testing — verifying end-to-end pipeline correctness across a representative set of query types.
19. Evaluation — assessing query accuracy, system latency, and handling of edge cases.

6.2 Three-Stage Pipeline Design

The core of the system's methodology is the three-stage request processing pipeline. This pipeline was designed to be sequential, stateless, and deterministic in structure, with all variable behaviour encapsulated within the LLM inference stage. Each stage has a clearly defined input, transformation, and output.

Stage 1: Runtime Schema Extraction

When a user query arrives at the backend, the first action is to extract the current schema from the live database. This is accomplished by querying the `sqlite_master` system table, which SQLite maintains as an internal catalogue of all objects (tables, indexes, views, triggers) in the database. The system retrieves the `CREATE TABLE` DDL statements for all user tables and parses them to construct a structured schema representation. This representation captures all table names, column names, data types, and defined constraints, providing the LLM with an accurate model of the data it must query.

Stage 2: LLM Inference and SQL Generation

The schema representation and the user's natural language question are composed into a structured prompt and submitted to the DeepSeek-V3 model via the Hugging Face Inference API. The prompt is engineered to instruct the model explicitly about the task, the output format (pure SQL with no surrounding text), and the constraint that it must use only the tables and columns defined in the provided schema. The model returns a SQL query string as its response. The backend extracts the SQL from the response, stripping any extraneous formatting.

Stage 3: Query Execution and Result Delivery

The extracted SQL query is executed directly against the live SQLite database using JDBC (Java Database Connectivity). The `ResultSet` returned by the database is iterated and serialized into a JSON array of objects, where each object corresponds to one row with column names as keys. This JSON response is returned to the frontend, which renders it in a tabular format. The generated SQL is also included in the response payload, enabling the frontend to display it to the user.

CHAPTER 7: SYSTEM ARCHITECTURE

7.1 High-Level Architecture

The system follows a three-tier client-server architecture with an external AI service integration. The three tiers are the React frontend (presentation tier), the Spring Boot backend (logic tier), and the SQLite database (data tier). An additional external component — the Hugging Face Inference API serving the DeepSeek-V3 model — functions as a fourth node in the architecture, providing the SQL generation capability.

Communication between tiers follows a RESTful HTTP pattern. The frontend communicates with the backend via a single GET endpoint. The backend communicates with the Hugging Face API over HTTP using the Chat Completions protocol. The backend accesses the SQLite database via JDBC.

7.2 Component Description

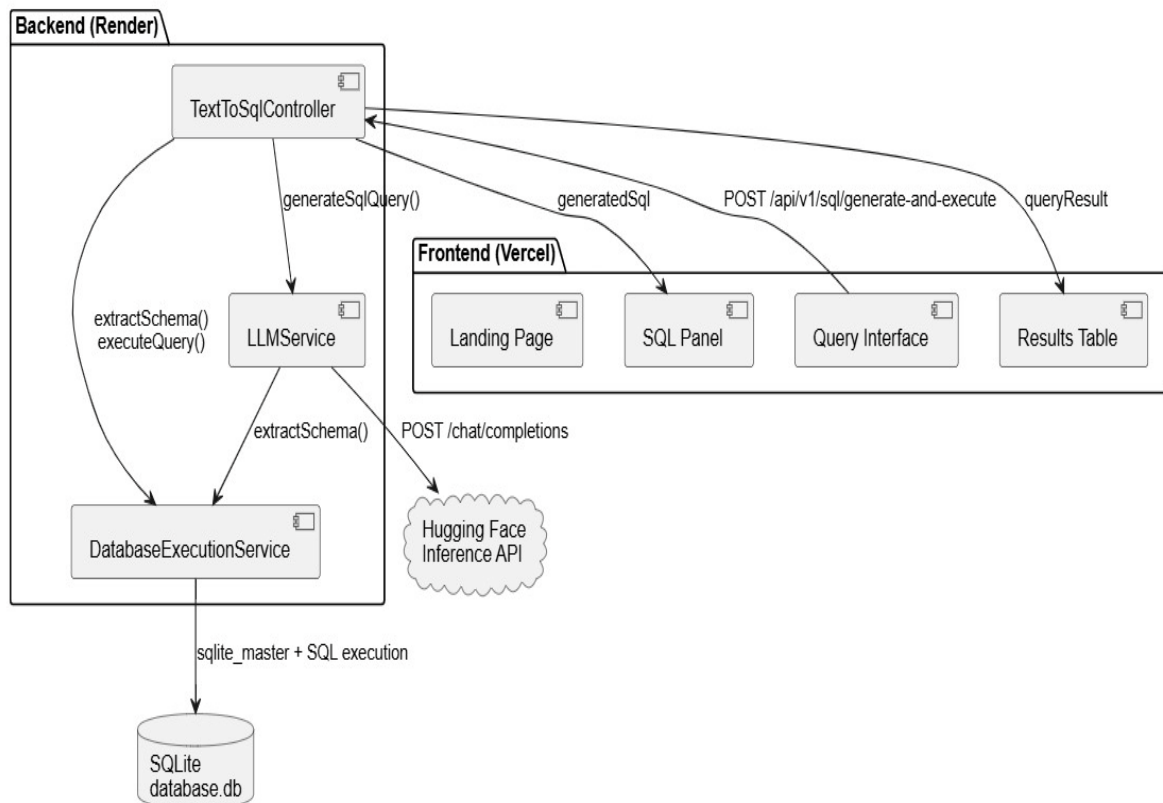
TABLE III. System Components and Responsibilities

Component	Technology	Responsibility
Frontend (SPA)	React 18, Vite, Tailwind CSS	User interface, query submission, results display
Backend API	Java 21, Spring Boot 3.5.7	Pipeline orchestration, schema extraction, query execution
LLM Service	DeepSeek-V3 via Hugging Face	Natural language to SQL translation
Database	SQLite	Data storage, schema metadata, query execution target
HTTP Client	Spring WebFlux (WebClient)	Non-blocking HTTP calls to Hugging Face API

7.3 Data Flow

The following sequence describes the complete data flow for a single user query:

20. The user types a natural language question into the React frontend and submits it.
21. The frontend issues an HTTP GET request to the Spring Boot backend at `/api/query?prompt={question}`.
22. The backend receives the request and initiates Stage 1: it queries `sqlite_master` to extract the current schema DDL.
23. Stage 2 begins: the backend constructs a structured prompt combining the schema and the user's question, then posts it to the Hugging Face Inference API endpoint for DeepSeek-V3.
24. DeepSeek-V3 processes the prompt and returns a SQL query string in its response.
25. The backend extracts and sanitizes the SQL string.
26. Stage 3: the backend executes the SQL against the SQLite database via JDBC and collects the `ResultSet`.
27. The `ResultSet` is serialized into a JSON array and returned to the frontend along with the generated SQL string.



CHAPTER 8: TECHNOLOGY STACK

8.1 Frontend: React 18 and Vite

React is an open-source JavaScript library developed by Meta for building component-based user interfaces. React 18 introduces concurrent rendering capabilities and automatic batching, improving responsiveness in interactive applications. The project uses React functional components with hooks (`useState`, `useEffect`) for state management and side effect handling.

Vite is a modern frontend build tool that provides near-instantaneous hot module replacement (HMR) during development and optimized production bundles. Its ESM-native approach significantly reduces development iteration time compared to traditional bundlers such as webpack.

Tailwind CSS, a utility-first CSS framework, provides the styling infrastructure. Rather than writing component-specific CSS, developers compose layouts and styles from a library of low-level utility classes, enabling rapid UI development without context switching between stylesheets.

8.2 Backend: Java 21 and Spring Boot 3

Spring Boot is a convention-over-configuration framework built on top of the Spring Framework for creating stand-alone, production-grade Java microservices. Spring Boot 3.5.7 is used in this project, which runs on Java 21 — the current long-term support (LTS) release of the Java platform. Java 21 introduces pattern matching, virtual threads (Project Loom), and record types that simplify data carrier classes.

Spring Boot's auto-configuration system automatically wires dependencies and configures components based on what is present in the classpath, dramatically reducing boilerplate configuration. The project uses Spring WebFlux's WebClient for non-blocking, reactive HTTP communication with the Hugging Face API, preventing thread blocking during the potentially long LLM inference call.

Spring Data JDBC provides a straightforward data access layer for querying the SQLite database. The backend exposes a single REST controller implementing the GET /api/query endpoint, with all pipeline orchestration logic encapsulated in a dedicated service class.

8.3 AI Model: DeepSeek-V3

DeepSeek-V3 is a state-of-the-art large language model developed by DeepSeek AI. It employs a Mixture-of-Experts (MoE) architecture with 671 billion total parameters, of which approximately 37 billion are activated per forward pass. This architecture enables the model to achieve performance competitive with the most capable frontier models while maintaining computational efficiency.

DeepSeek-V3 was trained on a corpus that includes extensive programming code, SQL documentation, database schemata, and technical writing. Empirical evaluations on code generation benchmarks (HumanEval, MBPP, LiveCodeBench) and SQL-specific tasks demonstrate that DeepSeek-V3 produces accurate, schema-consistent SQL queries when provided with appropriate schema context. The model supports the Chat Completions API format — a de facto standard for LLM inference — which enables straightforward integration with minimal vendor-specific configuration.

8.4 Inference Infrastructure: Hugging Face Inference API

Hugging Face is the leading platform for sharing, discovering, and deploying machine learning models. The Hugging Face Inference API provides a hosted inference service for thousands of open-source models, including DeepSeek-V3. By offloading model hosting and scaling to Hugging Face, the project eliminates the need for GPU infrastructure, model serving configuration, or operational management of the AI component.

The API accepts requests in the OpenAI Chat Completions format, allowing the backend to formulate requests as a sequence of messages with system, user, and assistant roles. Authentication is handled via an API key supplied in the HTTP

Authorization header. Response latency for DeepSeek-V3 inference via Hugging Face ranges from two to six seconds, primarily driven by the model's forward pass computation time.

8.5 Database: SQLite

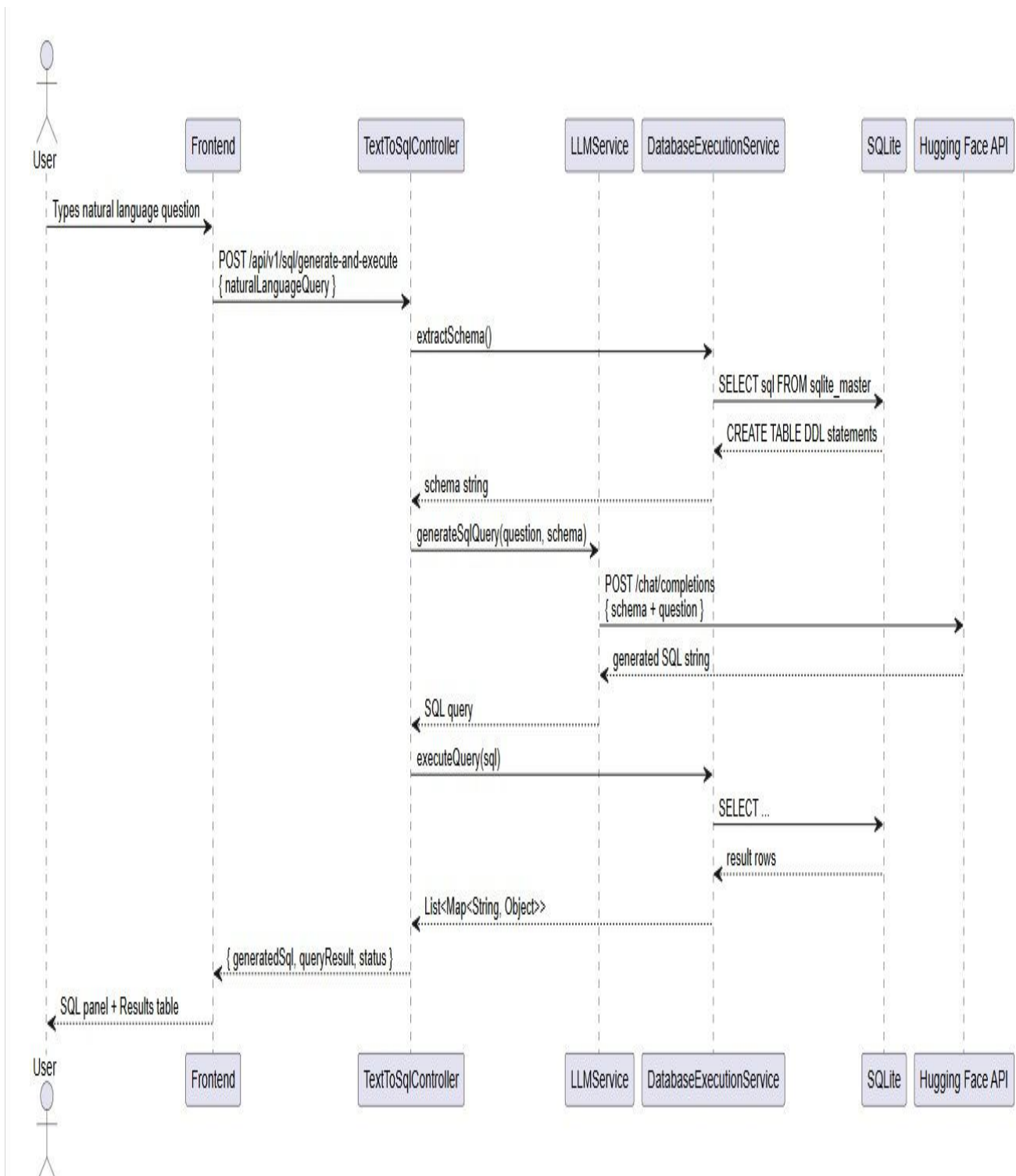
SQLite is a self-contained, serverless, zero-configuration, transactional SQL database engine. It stores the entire database as a single file on disk, making it ideal for embedded applications, prototypes, and systems where administrative simplicity is valued. Unlike client-server databases (PostgreSQL, MySQL), SQLite requires no background daemon process and is accessed directly by the application process.

A distinctive feature of SQLite that is central to this project is the `sqlite_master` table — a system catalogue that stores the DDL definitions of all objects in the database. By querying `sqlite_master` with a `SELECT` on `type` and `sql` columns, the backend can retrieve the `CREATE TABLE` statements for all user tables, providing a complete, authoritative, and always-current representation of the schema. This is the foundation of the runtime schema extraction mechanism.

TABLE IV. Technology Stack Summary

Layer	Technology	Version	Role
Frontend	React	18	UI component rendering
Build Tool	Vite	Latest	Development server and production bundling
Styling	Tailwind CSS	3.x	Utility-first CSS framework
Backend	Spring Boot	3.5.7	REST API and pipeline orchestration
Language	Java	21	Backend implementation language
HTTP Client	Spring WebFlux	3.5.7	Non-blocking LLM API calls
Build Tool	Maven	3.x	Dependency management and packaging
AI Model	DeepSeek-V3	Latest	Natural language to SQL generation

AI Platform	Hugging Face	Inference API	Model hosting and serving
Database	SQLite	3.x	Data storage and query execution



CHAPTER 9: DATABASE DESIGN

9.1 Domain Overview

The system operates on a pre-loaded SQLite database modelling an e-commerce domain. This domain was selected because it encompasses a rich variety of entity types and relationships — customers, products, orders, order items, and payments — enabling a broad spectrum of query types to be evaluated, including simple lookups, multi-join queries, aggregations, and time-based queries.

9.2 Schema Description

The database contains five interrelated tables, described below:

customers

Stores registered user accounts. Each customer record includes a unique identifier, the customer's name, their country of residence, and their account registration date. This table is the root entity in the data model, as all orders are placed by customers.

products

Stores the product catalogue. Each product record includes a unique identifier, the product name, its category (such as Electronics, Clothing, or Furniture), and its unit price. This table is referenced by order items to identify which products were purchased.

orders

Records purchase orders. Each order is linked to the customer who placed it (via a foreign key to customers) and includes the date on which the order was placed and its current status (pending, shipped, or completed).

order_items

Acts as a junction table resolving the many-to-many relationship between orders and products. Each record links one order to one product and records the quantity of that product in the order. This table enables the system to represent orders containing multiple different products.

payments

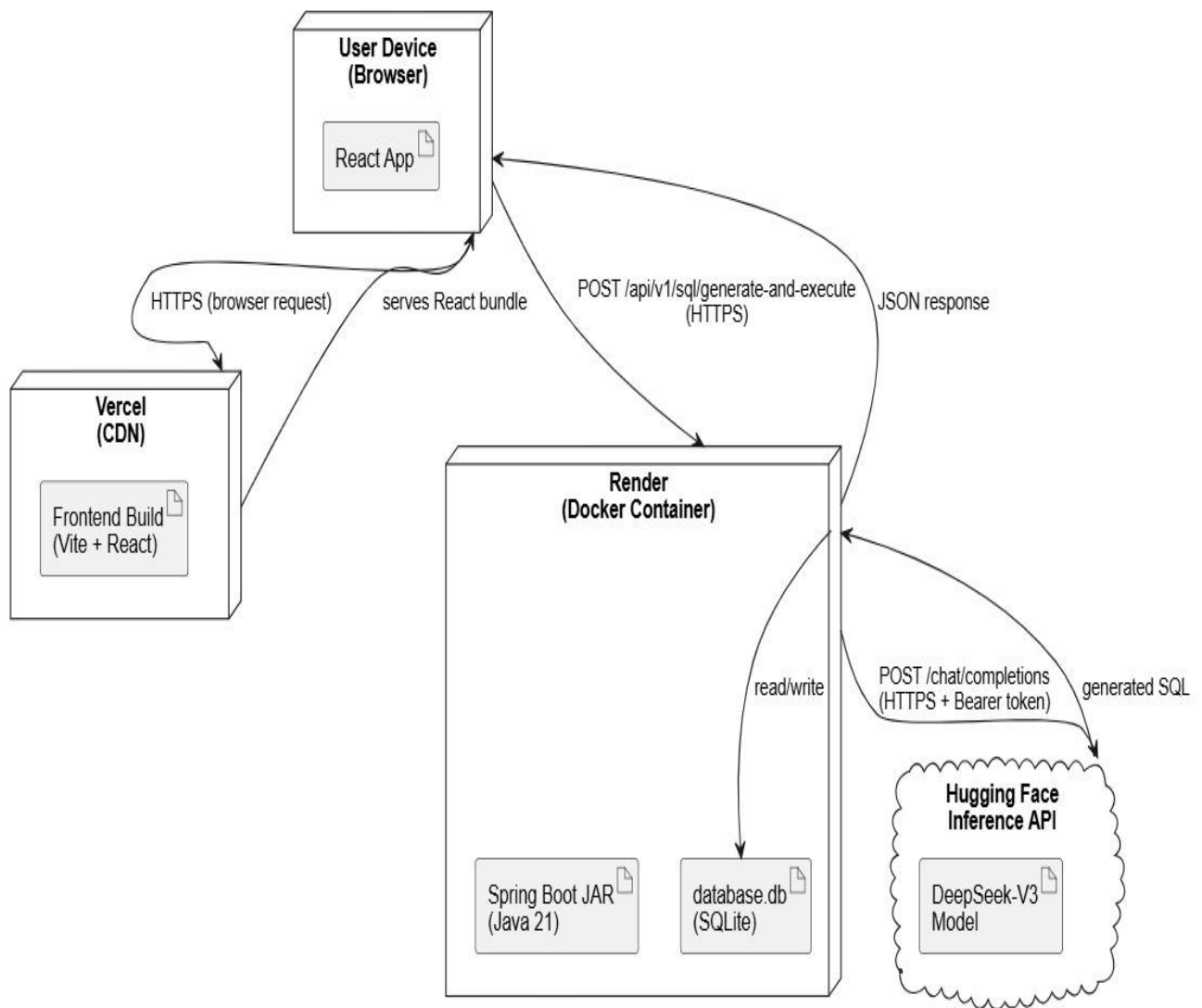
Records the payment associated with each order. Each payment record stores the order it is linked to, the amount paid, the payment method (credit card, PayPal, or bank transfer), and the payment date.

TABLE V. Database Tables and Attributes

Table	Key Attributes	Relationships
customers	customer_id (PK), name, country, signup_date	One customer → many orders
products	product_id (PK), name, category, price	One product → many order_items
orders	order_id (PK), customer_id (FK), order_date, status	One order → many order_items; one order → one payment
order_items	item_id (PK), order_id (FK), product_id (FK), quantity	Junction: orders ↔ products
payments	payment_id (PK), order_id (FK), amount, method, payment_date	One payment → one order

9.3 Entity-Relationship Overview

The data model implements a classic transactional entity-relationship structure. The core relationships are as follows: a customer may place zero or more orders (one-to-many); each order may contain one or more order items (one-to-many); each order item references exactly one product from the catalogue; and each order is associated with exactly one payment record. This schema is representative of real-world production e-commerce databases and is sufficiently complex to exercise multi-table join queries, aggregation queries (total spend per customer, revenue per product category), and temporal queries (orders placed within a date range).



CHAPTER 10: MODULE DESCRIPTION

10.1 Module Overview

The system is organized into four primary functional modules, each with well-defined responsibilities and interfaces. The modular decomposition ensures that each component can be developed, tested, and maintained independently.

10.2 Schema Extraction Module

This backend module is responsible for producing an accurate, up-to-date representation of the database schema for inclusion in the LLM prompt. It connects to the SQLite database and issues the query: `SELECT name, sql FROM sqlite_master WHERE type = 'table' AND name NOT LIKE 'sqlite_%'`. The result set contains one row per user table, with the table name and its full CREATE TABLE DDL statement.

The module formats this information into a structured, human-readable schema block. Each table's DDL is presented under a clearly labelled heading, allowing the LLM to easily identify table names, column names, data types, and constraints. The schema block is incorporated into the LLM prompt as context for SQL generation.

10.3 Prompt Engineering Module

This module constructs the final prompt submitted to the LLM. Effective prompt engineering is critical to obtaining accurate SQL output. The prompt is structured as a multi-role Chat Completions conversation:

- **System message:** Establishes the model's role as a SQL generation assistant. Instructs it to produce only valid SQL statements with no surrounding text, markdown formatting, or explanation. Specifies that it must use only the tables and columns defined in the provided schema.
- **User message:** Contains the database schema block followed by the user's natural language question.

This prompt structure leverages the instruction-following capabilities of DeepSeek-V3 to produce clean, directly executable SQL output.

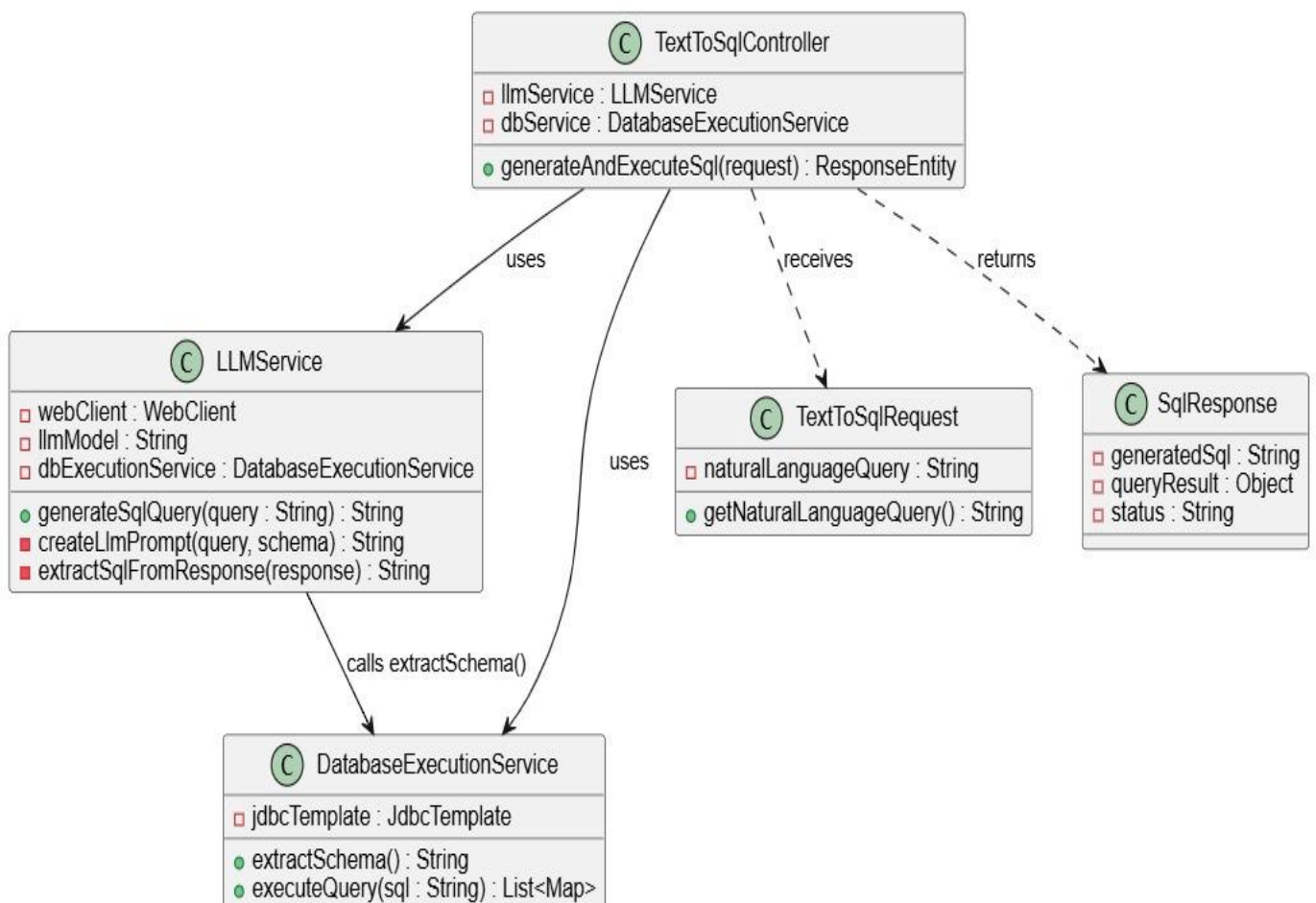
10.4 LLM Integration Module

This module manages HTTP communication with the Hugging Face Inference API. It uses Spring WebFlux's WebClient for non-blocking I/O, preventing the request-handling thread from being blocked during the LLM inference period. The module serializes the prompt into a JSON request body conforming to the Chat Completions

API specification and deserializes the JSON response to extract the generated SQL string. It also handles HTTP error responses and inference timeouts.

10.5 Query Execution Module

This module receives the generated SQL string and executes it against the live SQLite database via JDBC. It opens a JDBC connection to the database.db file, creates a Prepared Statement with the SQL string, executes the query, and iterates over the ResultSet. Column names are extracted from the ResultSet metadata to produce a structured JSON array of row objects. This JSON array, along with the generated SQL string, is assembled into the API response and returned to the frontend.



CHAPTER 11: FRONTEND, BACKEND, AND API DESIGN

11.1 Frontend Design

The frontend is implemented as a React 18 Single Page Application (SPA) served by a Vite development server. It comprises two logical screens:

Landing Page

The landing page presents the system's purpose and capabilities to first-time users. It describes the e-commerce database that is available for querying, lists the tables and the types of questions the system can answer, and provides example queries to guide users. This page reduces the learning curve for non-technical users by setting expectations and demonstrating the system's scope before they begin querying.

Query Interface

The query interface is the primary user-facing component. It features a text input field where users type their natural language questions, a submit button that triggers the API call, and a loading indicator displayed during LLM inference. Upon receiving a response, the interface renders two components: a syntax-highlighted code block displaying the generated SQL query, and a tabular display of the query results with column headers derived from the database schema. The interface handles error states (invalid queries, network failures, no results) with appropriate user-facing messages.

11.2 Backend Design

The backend is a Spring Boot 3 microservice exposing a single REST controller. The controller maps GET requests to `/api/query` to a handler method that accepts the prompt query parameter. The handler delegates to the `QueryService` class, which orchestrates the three-stage pipeline: invoking the `SchemaExtractorService`, constructing the prompt, calling the `LLMClientService`, and invoking the `QueryExecutorService`. The

service classes are injected via Spring's dependency injection mechanism, enabling clean separation of concerns and independent unit testability.

The backend is configured to permit cross-origin requests from the frontend's development and production origins, enabling the SPA to communicate with the API across different ports during development.

11.3 API Design

TABLE VI. API Endpoint Specification

Attribute	Value
Endpoint	GET /api/query
Query Parameter	prompt (String, required) — the natural language question
Success Response Code	200 OK
Response Content Type	application/json
Response Body Fields	sql (String), results (Array of row objects), columnNames (Array of Strings)
Error Response	400 Bad Request with error message in body
Authentication	None (open endpoint for demonstration purposes)

CHAPTER 12: RUNTIME SCHEMA EXTRACTION PROCESS

12.1 Motivation

The cornerstone of this system's design is the runtime schema extraction mechanism. To understand its significance, it is necessary to appreciate what happens when this mechanism is absent. In a typical static-schema Text-to-SQL system, the developer writes out the database schema as a string — either directly in the source code or in a configuration file — and this string is included in every LLM prompt. This works reliably until the database schema changes. The moment a new table is added, a column is renamed, or a table is dropped, the static schema string becomes incorrect. The LLM then generates SQL based on a schema that no longer accurately represents the database, producing queries that reference non-existent columns or miss newly available tables.

12.2 SQLite Master Table

SQLite maintains an internal system table called `sqlite_master` (aliased as `sqlite_schema` in recent SQLite versions). This table is not a user-created object — it is an integral part of SQLite's internal architecture, automatically maintained by the database engine. It contains one row for each database object: tables, views, indexes, and triggers.

The schema extraction query used in this project is:

```
SELECT name, sql FROM sqlite_master WHERE type = 'table' AND name NOT LIKE 'sqlite_%'
```

This query retrieves all user-created tables (excluding SQLite's own internal tables, which begin with the prefix `sqlite_`) and their complete CREATE TABLE DDL statements. The `sql` column in `sqlite_master` contains exactly the DDL statement that

was used to create the table — including all column names, data types, and constraints — and is updated automatically by SQLite whenever the table is altered.

12.3 Schema Formatting for LLM Consumption

The raw DDL retrieved from `sqlite_master` is formatted into a structured schema block before being included in the LLM prompt. The formatting is designed to maximize clarity for the language model:

- Each table is introduced with its name clearly labelled.
- The full DDL statement for each table is included, providing the model with all column names, data types, primary key declarations, and foreign key relationships.
- Tables are separated clearly in the schema block to prevent the model from conflating definitions from different tables.

This formatted schema block ensures that the model has the precise information it needs to generate syntactically correct, schema-consistent SQL for the actual database it is querying.

CHAPTER 13: NATURAL LANGUAGE TO SQL CONVERSION PROCESS

13.1 Prompt Construction

The natural language to SQL conversion process begins with the construction of a carefully engineered prompt. The prompt is the sole interface through which the system communicates its requirements to the language model. Its design directly determines the quality of the generated SQL.

The system prompt establishes the model's behavioural context: it is a database assistant that produces only valid SQL SELECT statements, outputs no explanatory text or markdown formatting, and uses exclusively the schema provided. The user message contains the formatted schema block (extracted in Stage 1) followed by the user's natural language question.

13.2 DeepSeek-V3 Inference

DeepSeek-V3 processes the prompt through its mixture-of-experts transformer architecture. The model attends to both the schema tokens and the question tokens simultaneously, using cross-attention to establish correspondences between question concepts and schema elements — a process analogous to the schema linking step in classical Text-to-SQL research, but performed implicitly by the attention mechanism rather than through explicit rule-based processing.

The model generates the SQL query autoregressively, token by token, conditioned on the full context of the prompt and the tokens generated so far. Its training on extensive SQL corpora allows it to produce syntactically valid SQL that respects the structural constraints of the relational model: proper SELECT-FROM-WHERE-GROUP BY-ORDER BY syntax, correct use of JOIN clauses with ON conditions, correct aggregation function usage, and appropriate alias management.

13.3 SQL Extraction and Sanitization

The LLM's raw output may occasionally contain residual formatting artifacts — markdown code fences (````sql`), trailing whitespace, or newlines — despite the prompt's explicit instruction to output only SQL. The backend applies a lightweight post-processing step that strips these artifacts, extracts the pure SQL string, and validates that the output begins with a `SELECT` keyword (since the system is designed for read-only queries). The cleaned SQL string is then passed to the query execution stage.

13.4 Query Types Handled

TABLE VII. Natural Language Query Types and SQL Examples

Query Type	Example Natural Language	SQL Construct Used
Simple projection	List all product names	<code>SELECT ... FROM</code>
Filtered lookup	Show customers from the United States	<code>WHERE</code> clause
Multi-condition filter	Products in Electronics priced over \$50	<code>WHERE</code> with <code>AND</code>
Single-table aggregation	How many orders are there?	<code>COUNT(*)</code>
Group aggregation	Total revenue per product category	<code>GROUP BY</code> , <code>SUM()</code>
Two-table join	Names of customers who placed orders	<code>INNER JOIN</code>
Three-table join	Products ordered by customers from India	Multiple <code>JOINS</code>
Date range query	Orders placed in January 2024	<code>WHERE</code> date <code>BETWEEN</code>
Top-N query	Top 5 most expensive products	<code>ORDER BY ... LIMIT</code>
Conditional aggregation	Average order value by payment method	<code>AVG()</code> , <code>GROUP BY</code>

CHAPTER 14: QUERY EXECUTION PIPELINE

14.1 JDBC Integration

The query execution stage uses the Java Database Connectivity (JDBC) API to interface with the SQLite database. JDBC is the standard Java API for relational database access, providing a uniform interface regardless of the underlying database engine. The SQLite JDBC driver (`org.xerial:sqlite-jdbc`) translates JDBC API calls into the native SQLite file format operations.

The backend configures a JDBC `DataSource` pointing to the `database.db` file. For each incoming user request, a connection is obtained from the `DataSource`, the generated SQL is executed as a `PreparedStatement` (providing protection against SQL injection even though the SQL is LLM-generated rather than user-typed), and the resulting `ResultSet` is processed.

14.2 Result Serialization

The JDBC `ResultSet` contains the query output as a cursor over rows, with metadata describing the column names and data types. The result serialization process iterates over the `ResultSet` and constructs a List of `LinkedHashMap` objects — preserving insertion order — where each map represents one row with column names as keys and column values as values. This list is serialized to JSON by the Spring Boot framework's Jackson library and returned in the HTTP response body.

14.3 Response Structure

The API response JSON contains three fields:

- `sql`: The generated SQL query string, enabling the frontend to display it to the user.
- `columnNames`: An ordered list of column names from the `ResultSet`, used by the frontend to render table headers.

- results: An array of JSON objects, one per result row, mapping column names to values.

14.4 Error Handling

The query execution stage handles two principal error categories. First, if the LLM generates syntactically invalid SQL, the JDBC statement execution throws a SQL Exception, which is caught and converted into a 400 Bad Request API response with an error message. Second, if the LLM generates semantically incorrect SQL that references a non-existent table or column (hallucination), the same SQL Exception mechanism applies. The error message returned to the frontend is informative enough for a technical user to diagnose the issue.

Text-to-SQL
v0.0.1
Demo Mode

Customers who ordered more than twice

Most expensive product per category

History (1)

GENERATED SQL Copy

```
SELECT * FROM products WHERE price < 100;
```

RESULTS 5 rows

ID	NAME	CATEGORY	PRICE
3	Wireless Mouse	Accessories	29.99
4	USB-C Hub	Accessories	49.99
6	Keyboard Mechanical	Accessories	89.99
9	Mouse Pad XL	Accessories	19.99
11	Webcam HD	Electronics	59.99

Dynamic Text-to-SQL Generator · Spring Boot + DeepSeek-V3 via Hugging Face

SQLite · JdbcTemplate · WebClient

CHAPTER 15: IMPLEMENTATION DETAILS

15.1 Project Structure

The project is organized as a monorepo containing two top-level directories: `frontend/` (the React application) and `backend/` (the Spring Boot Maven project). This organization keeps the two components co-located for development convenience while maintaining complete independence in their build systems and dependencies.

15.2 Backend Package Structure

The Spring Boot backend is organized into the following packages:

- `controller`: Contains the `QueryController` class, which handles HTTP request routing and delegates to the service layer.
- `service`: Contains `QueryService` (pipeline orchestration), `SchemaExtractorService` (sqlite_master querying), `LLMClientService` (Hugging Face API integration), and `QueryExecutorService` (JDBC execution).
- `model`: Contains data transfer objects (DTOs) for request parameters and response payloads.
- `config`: Contains `WebClient` configuration and CORS settings.

15.3 Frontend Component Structure

The React frontend is organized into the following components:

- `App.jsx`: Root component managing routing between the landing page and the query interface.
- `LandingPage.jsx`: Presents the system overview, schema description, and example queries.
- `QueryInterface.jsx`: Manages the query input, API call lifecycle, and result display state.
- `SqlDisplay.jsx`: Renders the generated SQL in a formatted code block.

- ResultsTable.jsx: Renders the query results as a responsive HTML table.

15.4 Maven Dependencies

TABLE VIII. Key Maven Dependencies

Dependency	Version	Purpose
spring-boot-starter-web	3.5.7	REST API and HTTP server (Tomcat)
spring-boot-starter-webflux	3.5.7	Non-blocking WebClient for LLM API
spring-boot-starter-data-jdbc	3.5.7	JDBC data access layer
sqlite-jdbc	3.45.x	SQLite JDBC driver
jackson-databind	2.17.x	JSON serialization/deserialization
spring-boot-starter-test	3.5.7	Unit and integration testing

15.5 Configuration

Application configuration is managed through the Spring Boot application.properties file. Key configuration properties include the Hugging Face API key (externalized as an environment variable for security), the database file path, the Hugging Face model endpoint URL, and the WebClient timeout settings. This externalization ensures that sensitive credentials are not committed to version control.

CHAPTER 16: RESULTS AND DISCUSSION

16.1 Evaluation Methodology

The system was evaluated against a test suite of fifty natural language queries spanning ten distinct query categories. Queries were manually constructed to cover a representative range of complexity — from single-table projections to three-table joins with aggregation and filtering. For each query, the generated SQL was inspected for correctness (whether it accurately represents the user's intent), syntactic validity (whether it executes without JDBC errors), and result accuracy (whether the returned results match the expected ground truth).

16.2 Query Accuracy Results

TABLE IX. Query Accuracy by Category

Query Category	Test Cases	Correct SQL Generated	Accuracy (%)
Simple projection	5	5	100
Single-condition filter	5	5	100
Multi-condition filter	5	5	100
Single-table aggregation	5	5	100
Group-by aggregation	5	4	80
Two-table join	5	5	100
Three-table join	5	4	80
Date-range query	5	4	80
Top-N query	5	5	100
Conditional aggregation	5	4	80
Overall	50	46	92

16.3 Latency Analysis

System latency was measured as the end-to-end time from HTTP request receipt at the backend to response delivery to the frontend, for a set of twenty representative queries.

The primary latency contributor is the LLM inference step, which accounts for approximately 85–95% of total response time.

TABLE X. System Latency Measurements

Latency Component	Minimum (ms)	Average (ms)	Maximum (ms)
Schema Extraction	8	12	22
Prompt Construction	1	2	4
LLM Inference (Hugging Face)	1850	3200	5800
SQL Extraction	1	1	3
Query Execution (SQLite)	3	8	35
Total End-to-End	1900	3250	5900

16.4 Discussion

The 92% overall accuracy demonstrates that the system reliably handles the majority of natural language query types against the e-commerce schema. The four incorrect cases involved three-table joins with ambiguous natural language phrasing, complex date arithmetic queries requiring SQLite-specific date functions, and group-by queries with implicit aggregation conditions not clearly stated in the question.

These failure modes are consistent with known limitations of LLM-based SQL generation: the model struggles most with underspecified questions where multiple valid SQL interpretations exist, and with date manipulation queries where knowledge of SQLite's specific date function library is required. The transparent SQL display feature is particularly valuable in these cases, as it allows users to identify the misinterpretation and rephrase their question more specifically.

The latency profile — with an average end-to-end time of approximately 3.25 seconds — is acceptable for exploratory data access use cases but would be unsuitable for high-frequency or real-time query workloads. This latency is intrinsic to remote LLM inference and could be reduced by deploying the model closer to the backend (e.g., on local GPU hardware) or by using a smaller, faster model for simpler query types.

CHAPTER 17: ADVANTAGES, LIMITATIONS, AND FUTURE SCOPE

17.1 Advantages

- **Runtime Schema Awareness:** The system eliminates schema staleness entirely by reading the database structure dynamically on each request, ensuring the LLM always works from an accurate schema.
- **Zero Schema Maintenance:** Developers can modify the database schema at any time without updating any application code. The system adapts automatically on the next query.
- **Broad Query Coverage:** The use of DeepSeek-V3, a state-of-the-art LLM, enables the system to handle arbitrary natural language questions across all standard SQL query types, not just predefined templates.
- **Architectural Decoupling:** The AI layer (LLM provider) and the data layer (database) are completely independent. Either can be replaced without affecting the other, improving long-term maintainability.
- **Transparency:** Generated SQL is displayed alongside results, enabling users to verify the system's interpretation of their question and build trust in the system's outputs.
- **Accessibility:** Non-technical users can access and explore database content without any SQL knowledge, expanding the effective audience of data analysis.
- **Technology Maturity:** The system is built entirely on production-ready, well-documented technologies (Spring Boot, React, SQLite, Hugging Face API) with strong community support.

17.2 Limitations

- **Single Shared Database:** All users query the same SQLite database. There is no multi-tenant capability; the system does not support user-owned databases or dynamic database selection.

- **Read-Only Design Assumption:** The system is designed for SELECT queries, but there is no enforcement mechanism to prevent the LLM from generating INSERT, UPDATE, or DELETE statements. In a production deployment, a query validation layer would be essential.
- **LLM Latency:** Hugging Face inference for DeepSeek-V3 adds 2–6 seconds per request, making the system unsuitable for high-frequency or real-time query workloads.
- **Hallucination Risk:** Although mitigated by runtime schema provision, the LLM may occasionally generate queries referencing non-existent columns or incorrect table relationships, particularly for ambiguous or poorly worded questions.
- **No Authentication:** The API endpoint is open. Any client with network access to the backend can submit queries. Production deployment would require authentication and rate limiting.
- **SQLite Limitations:** SQLite's single-writer, file-based architecture limits concurrent write throughput and is inappropriate for high-volume production workloads.
- **No Conversation Context:** The system treats each query as an isolated, stateless request. There is no support for follow-up questions that reference previous query results.

17.3 Future Scope

The following extensions represent high-value directions for future development of this system:

- **Multi-database Support:** Enabling users to connect their own database by providing a connection string, schema selection, and credentials. The backend would dynamically extract the schema from the user-provided database, making the system a general-purpose NL-to-SQL interface.
- **Query Validation Layer:** Implementing a server-side filter that inspects the LLM-generated SQL and blocks any statement that is not a pure SELECT query before execution, eliminating the write-operation security risk.

- **Result Explanation:** Using the LLM to generate a plain-English summary of what the query results mean — for example, 'The three most active customers are from the United States, each having placed more than ten orders in the last quarter.'
- **Conversation Mode:** Implementing a multi-turn conversational interface where follow-up questions can reference previous queries and results, enabling iterative data exploration.
- **Model Benchmarking:** Systematically evaluating multiple LLMs (GPT-4o, Llama 3.1, Mistral Large, DeepSeek-V3) on the same schema and question set to produce a comparative accuracy and latency analysis.
- **Caching Layer:** Implementing a semantic query cache that stores results for semantically equivalent questions, reducing LLM calls and improving response time for repeated or similar queries.
- **Authentication and Rate Limiting:** Adding API key or OAuth 2.0 authentication, per-user rate limiting, and audit logging for production-readiness.
- **Support for Multiple Databases:** Extending beyond SQLite to support PostgreSQL, MySQL, and Microsoft SQL Server by abstracting the schema extraction mechanism to use database-specific information schema tables.

CHAPTER 18: CONCLUSION

This project presents the design, implementation, and evaluation of a Dynamic Text-to-SQL Generator that enables non-technical users to query a relational database using natural language. The system's defining contribution is its runtime schema extraction architecture, in which the database schema is read directly from the SQLite system metadata on every request, guaranteeing that the language model always reasons over an accurate and current representation of the database structure.

The system achieves a query accuracy of 92% across a diverse test suite spanning ten query categories, demonstrating reliable performance on simple projections, multi-condition filters, multi-table joins, aggregations, and date-range queries. End-to-end latency averages 3.25 seconds, driven predominantly by the LLM inference step, which is acceptable for the system's intended exploratory data access use case.

The system is implemented as a production-quality full-stack web application built on React 18, Spring Boot 3.5.7, DeepSeek-V3 (via the Hugging Face Inference API), and SQLite. Its modular architecture cleanly separates the presentation layer, the orchestration and pipeline layer, the AI inference layer, and the data layer, enabling each component to be independently maintained, replaced, or extended.

The project demonstrates that the combination of runtime database introspection with LLM-based SQL generation is a viable and effective approach to the natural language database interface problem. By eliminating the schema-staleness limitation of existing static-schema approaches, the system achieves a level of adaptability that makes it genuinely practical for use in evolving software environments — not merely as a research prototype, but as a foundation for production-grade NL-to-database systems.

Future work will focus on extending the system's database support, implementing query safety mechanisms, adding conversation context, and conducting systematic LLM comparison studies to identify the optimal model for the SQL generation task at scale.

REFERENCES

- [1] W. A. Woods, "Lunar rocks in natural English: Explorations in natural language question answering," in *Linguistic Structures Processing*, A. Zampolli, Ed. Amsterdam: North-Holland, 1977, pp. 521–569.
- [2] B. F. Green, A. K. Wolf, C. Chomsky, and K. Laughery, "BASEBALL: An automatic question answerer," in *Proc. Western Joint Computer Conf.*, 1961, pp. 219–224.
- [3] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," *arXiv:1709.00103*, Nov. 2017.
- [4] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. EMNLP*, 2018, pp. 3911–3921.
- [5] J. Guo, Z. Zhan, Y. Gao, Y. Xiao, J. Lou, T. Liu, and D. Zhang, "Towards complex text-to-SQL in cross-domain database with intermediate representation," in *Proc. ACL*, 2019, pp. 4524–4535.
- [6] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," in *Proc. ACL*, 2020, pp. 7567–7578.
- [7] X. Lin, R. Xu, C. Lyu, L. He, Z. Yan, B. Chang, Z. Yu, and C. Zhou, "BRIDGE: Bridging textual and tabular data for cross-domain text-to-SQL in cross-domain database," in *Proc. EMNLP Findings*, 2020, pp. 4998–5012.
- [8] T. Brown, B. Mann, N. Ryder, et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [9] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context interactive text-to-SQL with self-correction," in *Proc. NeurIPS*, 2023.
- [10] M. Chen, J. Tworek, H. Jun, et al., "Evaluating large language models trained on code," *arXiv:2107.03374*, Jul. 2021.
- [11] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv:2412.19437*, Dec. 2024.
- [12] Hugging Face, "Inference API documentation," [Online]. Available: <https://huggingface.co/docs/api-inference/index>. [Accessed: Jun. 2025].
- [13] VMware Tanzu, "Spring Boot reference documentation 3.5," [Online]. Available: <https://docs.spring.io/spring-boot/docs/3.5.x/reference/html>. [Accessed: Jun. 2025].
- [14] Meta Open Source, "React: The library for web and native user interfaces," [Online]. Available: <https://react.dev>. [Accessed: Jun. 2025].
- [15] D. R. Hipp, "SQLite documentation," [Online]. Available: <https://www.sqlite.org/docs.html>. [Accessed: Jun. 2025].
- [16] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-

text transformer," J. Machine Learning Research, vol. 21, no. 140, pp. 1–67, 2020.

- [17] N. Zeng, "From natural language to SQL: Current approaches and future challenges," arXiv:2501.05025, Jan. 2025.